Overview
Welcome to Semantica

Core Concepts
Core Concepts
Modules
Choose the Right Module
Glossary

Guides
Architecture
Learning More
GraphRAG: Graph-Augmented Retrieval
Ingest Anything
LLM Integrations
Pipeline Builder
Multi-Agent Systems
Decision Intelligence
Context Graphs
Agent Memory
Provenance & Audit Trails
Semantic Extraction
Ontology Management
Reasoning & Rules
Export & Serialization
Deduplication & Entity Merging
MCP Server
SHACL Validation
Conflict Detection & Resolution
Change Management & Versioning
Policy Engine
Visualization
Distance Intelligence
Graph Analytics

Integrations
Agno Integration
CrewAI Integration
LangChain Integration
Docling Integration
Snowflake Integration
Databricks Integration
Salesforce Integration

Graph Stores
Apache AGE Graph Store

Vector Stores
pgvector Store

FAQ
FAQ

Community
Community
Community Projects
Contributing
Governance
Citation
License

On this page

What Is SHACL Validation?
Why Use SHACL Validation?
When To Use / When Not To Use
Key Terms Explained
Typical Workflow
Universal Example: Employee & Department
Step 1 — Build the ontology from your merged graph
Step 2 — Generate SHACL shapes from the ontology
Step 3 — Inject domain constraints
Step 4 — Run validation and read the report
Step 5 — Understand the violations
Step 6 — Auto-remediate common violations
Common Pitfalls
Domain Examples
Resource limits
Using SHACL validation as a CI/CD gate
Related Guides

Guides

# SHACL Validation

Generate W3C SHACL shapes from OWL ontologies, validate RDF knowledge graphs against structural and semantic constraints, and surface structured violation reports.

## What Is SHACL Validation?

SHACL (Shapes Constraint Language) is a standard for validating graph-based data. While an ontology defines the conceptual *schema* (the "what" exists in your domain), SHACL defines the structural *rules and constraints* (the "how" it should be structured).

In Semantica, `SHACLGenerator` produces constraint rules (shapes) based on your ontology, and the public `run_shacl_validation` function evaluates your actual data against these rules. If a node violates a rule (e.g., missing a required property or using the wrong datatype), a detailed violation report is generated. The historical `_run_pyshacl` name remains available as a compatibility alias.

## Why Use SHACL Validation?

Data validation is critical before running analytics, exporting data, or feeding it into production models. SHACL acts as a **data quality gate** that ensures your graph data is structurally sound. Use it to catch:

- Missing required properties (e.g., a customer without an email address).
- Datatype mismatches (e.g., a string where a number was expected).
- Cardinality breaches (e.g., a person with three primary addresses).

## When To Use / When Not To Use

- **When to Use**: You have a complex, interconnected knowledge graph and need to validate the *relationships* and structural integrity of the nodes across the graph. SHACL excels at ensuring that merged, highly connected data conforms to your business rules.
- **When NOT to Use**: If you are simply validating a flat JSON payload or a single incoming API request. For flat data or single records, use simpler, faster libraries like Pydantic or JSONSchema.

---

## Key Terms Explained

Before diving in, here are a few concepts you'll encounter:

- **RDF (Resource Description Framework)**: A standard way of representing data as a graph. It treats information as connected "triplets" (Subject → Predicate → Object).
- **OWL (Web Ontology Language)**: A language used to build ontologies. It defines the classes and properties that exist in your domain.
- **SHACL Shapes**: The actual validation rules. A "Shape" targets a specific class in your data (like `Person`) and defines the constraints it must follow (like "must have one birthdate").
- **Turtle (.ttl)**: A popular, human-readable file format for storing RDF graph data and SHACL shapes.

---

## Typical Workflow

A typical SHACL validation pipeline follows this lifecycle:

1. **Ontology**: Build an ontology representing your domain.
2. **SHACL Shapes**: Generate shapes from that ontology.
3. **Data Graph**: Prepare your knowledge graph.
4. **Validation**: Validate the knowledge graph against the SHACL shapes.
5. **Violation Report**: Analyze the report for errors.
6. **Remediation**: Fix the data or pipeline and re-validate.

---

## Universal Example: Employee & Department

Let's look at a simple, universally understood example: ensuring every `Employee` belongs to a `Department` and has an `employee_id`.

```python
from semantica.context import ContextGraph
from semantica.ontology import OntologyGenerator, SHACLGenerator, PropertyShape
from semantica.ontology import run_shacl_validation

# 1. Prepare your data graph
graph = ContextGraph()
graph.add_node("emp-1", "Employee", "Alice", employee_id="E001")
graph.add_node("emp-2", "Employee", "Bob") # Missing employee_id, will cause a

# 2. Build the ontology
ontology = (
    OntologyGenerator(base_uri="https://company.example.com/ontology/", min_occ
    .generate_from_graph(graph.to_dict(), name="CompanyOntology")
)

# 3. Generate SHACL Shapes
shacl_gen = SHACLGenerator(base_uri="https://company.example.com/shapes/", seve
shacl_graph = shacl_gen.generate(ontology)

# Inject mandatory constraints
BASE = "https://company.example.com/ontology/"
for ns in shacl_graph.node_shapes:
    if "Employee" in ns.target_class:
        ns.property_shapes.append(
            PropertyShape(path=f"{BASE}employee_id", min_count=1, severity="Vic
        )

# Serialize shapes to Turtle
shacl_ttl = shacl_gen.serialize(shacl_graph, format="turtle")

# 4. Prepare your RDF data graph
# (For validation, serialize your graph instances to RDF. Here we use a Turtle
data_ttl = """
@prefix ex: <https://company.example.com/ontology/> .

<http://example.org/emp-1> a ex:Employee ;
    ex:employee_id "E001" .

<http://example.org/emp-2> a ex:Employee .
"""

# 5. Run Validation
report = run_shacl_validation(data_ttl, shacl_ttl)

# 6. Analyze the Report
print(f"Graph conforms: {report.conforms}")
if not report.conforms:
    report.explain_violations() # Populates human-readable explanations
    for v in report.violations:
        print(f"Violation: {v.explanation}")
```

---

Now, let's explore the workflow in more depth.

## Step 1 — Build the ontology from your merged graph

SHACL shapes are derived from an ontology. If you already have one from a previous run, skip this step.

```python
from semantica.context import AgentContext, ContextGraph
from semantica.vector_store import VectorStore
from semantica.ontology import OntologyGenerator

graph = ContextGraph()
ctx   = AgentContext(
    vector_store=VectorStore(backend="faiss", dimension=768),
    knowledge_graph=graph,
    graph_expansion=True,
)

# Load the merged CTI data — in production this would be your full 12,000-node
ctx.store(
    [
        "APT29 is a Russian state-sponsored threat actor targeting NATO governm
        "CVE-2024-3400 is a critical vulnerability in PAN-OS exploited by APT2
        "HAMMERTOSS is a backdoor malware family used by APT29 for C2 over Twit
        "PAN-OS is a network operating system developed by Palo Alto Networks.'
    ],
    extract_entities=True,
    extract_relationships=True,
)

ontology = (
    OntologyGenerator(base_uri="https://cti.example.org/ontology/", min_occurre
    .generate_from_graph(graph.to_dict(), name="CyberOntology")
)

print(f"Classes inferred: {len(ontology.get('classes', []))}")
# Classes inferred: 4  → ThreatActor, Vulnerability, Malware, Platform
```

---

## Step 2 — Generate SHACL shapes from the ontology

`SHACLGenerator` produces a `SHACLGraph` with one `NodeShape` per OWL class.

```python
from semantica.ontology import SHACLGenerator

shacl_gen = SHACLGenerator(
    base_uri="https://cti.example.org/shapes/",
    include_inherited=True,   # propagate parent-class constraints to sub-class
    severity="Violation",     # default severity for all generated shapes
    quality_tier="standard",  # constraint strictness: "minimal" | "standard" |
)

shacl_graph = shacl_gen.generate(ontology)

print(f"Node shapes generated: {len(shacl_graph.node_shapes)}")
# Node shapes generated: 4  — one per class

for ns in shacl_graph.node_shapes:
    print(f"  {ns.target_class}  ({len(ns.property_shapes)} property constraint
# https://cti.example.org/ontology/ThreatActor   (2 property constraints)
# https://cti.example.org/ontology/Vulnerability  (3 property constraints)
# https://cti.example.org/ontology/Malware        (2 property constraints)
# https://cti.example.org/ontology/Platform       (1 property constraint)
```

The generated shapes tell you what the pipeline observed. They do not yet encode what your domain *requires*. The next section shows how to inject domain-specific mandatory constraints.

---

## Step 3 — Inject domain constraints

Add mandatory `PropertyShape` constraints the pipeline cannot infer from data alone.

```python
from semantica.ontology import PropertyShape

BASE = "https://cti.example.org/ontology/"

for node_shape in shacl_graph.node_shapes:

    if "Malware" in node_shape.target_class:
        # family is required — missing it causes a Violation
        node_shape.property_shapes.append(
            PropertyShape(
                path=f"{BASE}family",
                min_count=1,
                severity="Violation",
            )
        )
        # attribution_confidence is recommended — missing it causes a Warning
        node_shape.property_shapes.append(
            PropertyShape(
                path=f"{BASE}attribution_confidence",
                min_count=1,
                datatype="http://www.w3.org/2001/XMLSchema#float",
                severity="Warning",
            )
        )

    if "Vulnerability" in node_shape.target_class:
        # cvss_score is required by your detection rules
        node_shape.property_shapes.append(
            PropertyShape(
                path=f"{BASE}cvss_score",
                min_count=1,
                datatype="http://www.w3.org/2001/XMLSchema#float",
                severity="Violation",
            )
        )

    if "ThreatActor" in node_shape.target_class:
        # name is required; nation_state classification is recommended
        node_shape.property_shapes.append(
            PropertyShape(path=f"{BASE}name",         min_count=1, severity="Vi
        )
        node_shape.property_shapes.append(
            PropertyShape(path=f"{BASE}nation_state",  min_count=1, severity="W
        )

# Serialise the final shape graph to Turtle for reuse and version control
shacl_ttl = shacl_gen.serialize(shacl_graph, format="turtle")

with open("cti_shapes.ttl", "w") as f:
    f.write(shacl_ttl)

print("Shapes written to cti_shapes.ttl")
```

You can also construct shapes manually from scratch — useful when you need to express constraints the generator would never infer, such as a regex pattern on a CVE ID field:

```python
from semantica.ontology import NodeShape, PropertyShape, SHACLGraph

# Require CVE IDs to match the canonical NIST format
cve_id_shape = NodeShape(
    target_class="https://cti.example.org/ontology/Vulnerability",
    name="VulnerabilityShape",
    closed=False,
    severity="Violation",
    property_shapes=[
        PropertyShape(
            path="https://cti.example.org/ontology/cve_id",
            min_count=1,
            pattern=r"^CVE-\d{4}-\d{4,}$",   # e.g. CVE-2024-3400
            severity="Violation",
        ),
    ],
)
# Inject into the existing shacl_graph or build a standalone SHACLGraph
```

---

## Step 4 — Run validation and read the report

Serialize the graph to RDF, then run `run_shacl_validation` against the shapes.

```python
from semantica.ontology import run_shacl_validation

# Prepare your RDF data string (since export_rdf primarily exports structural m
# you typically serialize your custom data graph to Turtle using rdflib or simi
data_ttl = """
@prefix ex: <https://cti.example.org/ontology/> .

<http://example.org/malware-002> a ex:Malware .
<http://example.org/vuln-003> a ex:Vulnerability ;
    ex:cve_id "CVE24-3400" .
"""

# Run SHACL validation
report = run_shacl_validation(
    data_ttl,
    shacl_ttl,
    data_graph_format="turtle",
    shacl_format="turtle",
)

# High-level summary
print(f"Conforms    : {report.conforms}")
# Conforms    : False   ← at least one Violation found

print(f"Violations : {report.violation_count}")
# Violations : 3

print(f"Warnings    : {report.warning_count}")
# Warnings    : 2

print(report.summary())
# Graph does NOT conform: 3 violation(s).
```

The summary tells you something is wrong. Now drill into the details.

---

## Step 5 — Understand the violations

Each `SHACLViolation` identifies the node, property path, and fix required.

```python
if not report.conforms:
    # Populate plain-English explanations for every violation
    report.explain_violations()

    # Iterate and print the explanations
    for v in report.violations:
        print(v.explanation)
    # Node <http://example.org/malware-002> is missing required property
    #   <https://cti.example.org/ontology/family>. At least 1 value(s) are requ
    # Node <http://example.org/vuln-003> is missing required property
    #   <https://cti.example.org/ontology/cvss_score>. At least 1 value(s) are
    # Node <http://example.org/vuln-003> has value 'CVE24-3400' for
    #   <https://cti.example.org/ontology/cve_id> which does not match the requ

    # Iterate for programmatic triage
    for v in report.violations:
        print(f"VIOLATION  node={v.focus_node}")
        print(f"           path={v.result_path}")
        print(f"           rule={v.constraint}")
        print(f"           msg ={v.message}")
        if v.value:
            print(f"           val ={v.value}")
        if v.explanation:
            print(f"           fix ={v.explanation}")
        print()

    # Warnings are lower severity — review but do not block
    for w in report.warnings:
        print(f"WARNING  {w.focus_node}  {w.result_path}  {w.message}")
```

The output maps directly to remediation tasks: `malware-002` needs a `family` property added; `vuln-003` needs a `cvss_score` and its `cve_id` corrected to the canonical format.

## Step 6 — Auto-remediate common violations

Flag or patch nodes missing required properties, then re-validate to confirm.

```
# Parse the report into a dict for programmatic processing
report_dict = report.to_dict()

# Collect nodes missing the 'family' property
missing_family = [
    v["focus_node"]
    for v in report_dict.get("violations", [])
    if "family" in (v.get("result_path") or "")
]

print(f"Malware nodes missing 'family': {len(missing_family)}")
# In production: queue these for analyst enrichment or apply a default
# e.g. graph.update_node(node_id, {"family": "UNKNOWN — requires triage"})

# After remediation, re-run validation to confirm the fix
# (re-export the patched graph to Turtle first, then call run_shacl_validation
report2 = run_shacl_validation(patched_data_ttl, shacl_ttl)
print(f"Violations after remediation: {report2.violation_count}")
# Violations after remediation: 0
```

## Common Pitfalls

- **Assuming the ontology automatically enforces data quality**: `SHACLGenerator` generates shapes based on what it observes in the data. If your data is missing a field, the generator won't know it was mandatory unless you explicitly inject the constraint (as shown in Step 3).
- **Passing `ContextGraph` directly to SHACL validators**: The `run_shacl_validation` function expects an RDF string (like Turtle format), not a raw Python dictionary or `ContextGraph` object.
- **Forgetting RDF serialization**: You must serialize your graph (often via a temporary file using `export_rdf`) before validating it.
- **Treating validation as a one-time step**: Validation should be integrated as an automated step in your CI/CD pipeline or data ingestion flow, acting as a recurring gatekeeper rather than a one-off script.
- **Ignoring validation reports**: A graph that does not conform must be remediated. Failing to review the `violation_count` and address the issues negates the purpose of SHACL validation.
- **Validating `sh:class` / `sh:node` range checks on a property that declares `rdfs:range` with RDFS entailment on**: RDFS is an entailment rule, not a constraint. When pyshacl runs with `inference="rdfs"`, it infers the range class onto every object of the property, so class-based constraints on that property can never fail — the report says `conforms: True` on data that does not conform:

  ```
  from pyshacl import validate
  from rdflib import Graph

  data = Graph()
  data.parse(
      data="""
      @prefix ex: <https://example.org/ns#> .
      @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
      ex:contains rdfs:domain ex:Container ; rdfs:range ex:Item .
      ex:box a ex:Container ; ex:contains ex:notAnItem .
      ex:notAnItem a ex:Fish .
      """,
      format="turtle",
  )

  shapes = Graph()
  shapes.parse(
      data="""
      @prefix ex: <https://example.org/ns#> .
      @prefix sh: <http://www.w3.org/ns/shacl#> .
      ex:ContainerShape a sh:NodeShape ;
          sh:targetClass ex:Container ;
          sh:property [ sh:path ex:contains ; sh:class ex:Item ] .
      """,
      format="turtle",
  )

  for inference in ("none", "rdfs"):
      conforms, _, _ = validate(data, shacl_graph=shapes, inference=inference
      print(inference, conforms)
  # none  False   <- correct: notAnItem is a Fish, not an Item
  # rdfs  True    <- the entailment manufactured the type
  ```

  Mitigations: prefer not to declare `rdfs:range` on properties you intend to constrain with `sh:class`; when class membership is the thing under test, run validation without RDFS entailment (`inference="none"`); or express the check as a constraint the entailment cannot satisfy (for example a literal property constraint). Note the trade-off: with entailment off, `sh:targetClass` no longer reaches subclasses, so subclass hierarchies need explicit typing or inference-aware target selection. Semantica's own `run_shacl_validation` wrapper already calls pyshacl with `inference="none"`, so this pitfall only bites when calling `pyshacl.validate` directly with entailment enabled.
- **Trusting `conforms: True` without checking the inference mode**: an inference-enabled run can hide the exact violations the shapes were written to catch (see above). Record which inference mode validation ran under alongside the result, and re-run shape sets that contain `sh:class` / `sh:node` with entailment off before treating a pass as authoritative.

## Domain Examples

**Defense — CTI/Threat** Security — SOC/Incident Life Science — Clinical/Pharma Banking — R

A DoD CTI team enforces STIX-compatible constraints on a threat graph before sharing it with ISAC partners. Every `ThreatActor` must declare a `name` and every `Vulnerability` must carry a `cvss_score`. The validation gate runs automatically on each nightly sync.

```
from semantica.context import AgentContext, ContextGraph
from semantica.vector_store import VectorStore
from semantica.ontology import OntologyGenerator, SHACLGenerator, PropertyShape
from semantica.ontology import run_shacl_validation

graph = ContextGraph()
ctx   = AgentContext(
    vector_store=VectorStore(backend="faiss", dimension=768),
    knowledge_graph=graph,
    graph_expansion=True,
)

ctx.store([
    "APT29 is a Russian state-sponsored threat actor targeting NATO governments
    "CVE-2024-3400 is a critical PAN-OS vulnerability with CVSS 10.0, exploited
    "HAMMERTOSS is a backdoor malware family used by APT29 for C2 over Twitter
], extract_entities=True, extract_relationships=True)

ontology  = (
    OntologyGenerator(base_uri="https://cti.dod.mil/ontology/", min_occurrences
    .generate_from_graph(graph.to_dict(), name="CTIOntology")
)

shacl_gen   = SHACLGenerator(
    base_uri="https://cti.dod.mil/shapes/",
    include_inherited=True,
    severity="Violation",
)
shacl_graph = shacl_gen.generate(ontology)

# STIX-aligned mandatory fields
for ns in shacl_graph.node_shapes:
    if "ThreatActor" in ns.target_class:
        ns.property_shapes.append(
            PropertyShape(path="https://cti.dod.mil/ontology/name",         mir
        )
        ns.property_shapes.append(
            PropertyShape(path="https://cti.dod.mil/ontology/nation_state",  mi
        )
    if "Vulnerability" in ns.target_class:
        ns.property_shapes.append(
            PropertyShape(path="https://cti.dod.mil/ontology/cvss_score",   mir
        )

shacl_ttl = shacl_gen.serialize(shacl_graph, format="turtle")

# Prepare RDF data string
data_ttl = """
@prefix ex: <https://cti.dod.mil/ontology/> .

<http://example.org/apt29> a ex:ThreatActor .
<http://example.org/cve-2024-3400> a ex:Vulnerability .
<http://example.org/hammertoss> a ex:Malware .
"""

report = run_shacl_validation(data_ttl, shacl_ttl)
print(f"CTI graph conforms : {report.conforms}")
print(f"Violations         : {report.violation_count}")
print(f"Warnings           : {report.warning_count}")

if not report.conforms:
    report.explain_violations()
    for v in report.violations:
        print(v.explanation)
    # Blocks the nightly ISAC share until violations are resolved
```

## Resource limits

Live SHACL validation in the Explorer enforces four resource limits, all configurable through environment variables. When a limit trips, the error message names the variable that controls it.

| ENVIRONMENT VARIABLE | DEFAULT | WHAT IT BOUNDS |
|---|---|---|
| `SEMANTICA_MAX_SHACL_TURTLE_BYTES` | `262144` (256 KB) | Size of the submitted SHACL Turtle |
| `SEMANTICA_MAX_SHACL_TRIPLES` | `1000` | Triple count of the parsed shapes graph |
| `SEMANTICA_MAX_SHACL_TIMEOUT` | `15.0` | Validation timeout in seconds |
| `SEMANTICA_MAX_SHACL_CONCURRENCY` | `4` | Concurrent validations per process |

The first three are surfaced in the validation error message when exceeded; the concurrency limit applies as a semaphore and does not appear in responses.

## Using SHACL validation as a CI/CD gate

Call this function as a pre-publish gate; exit code 1 blocks the pipeline.

```
import sys
from semantica.ontology import OntologyGenerator, SHACLGenerator
from semantica.ontology import run_shacl_validation

def validate_before_publish(data_graph_str: str, ontology: dict) -> None:
    shacl_gen   = SHACLGenerator(base_uri="https://example.org/shapes/")
    shacl_graph = shacl_gen.generate(ontology)
    shacl_ttl   = shacl_gen.serialize(shacl_graph, format="turtle")

    report = run_shacl_validation(data_graph_str, shacl_ttl)

    if not report.conforms:
        print(f"Graph validation FAILED — {report.violation_count} violation(s
        report.explain_violations()
        for v in report.violations:
            print(v.explanation)
        sys.exit(1)

    print(f"Graph validation PASSED ({report.warning_count} warning(s))")
```

## Related Guides

- **Ontology Management** — generate the OWL ontology that SHACL shapes are derived from
- **Reasoning & Rules** — complement SHACL structural constraints with logical inference rules
- **Export & Serialization** — serialize graph data to Turtle/RDF/XML for `run_shacl_validation` input
- **Conflict Resolution** — detect and resolve data conflicts before SHACL validation
- **Change Management** — version-gate SHACL shapes alongside ontology versions

‹ MCP Server

Conflict Detection & Resolution ›

Powered by mintlify